

Skill-Centric Unified Job Search Aggregator System — Design Document

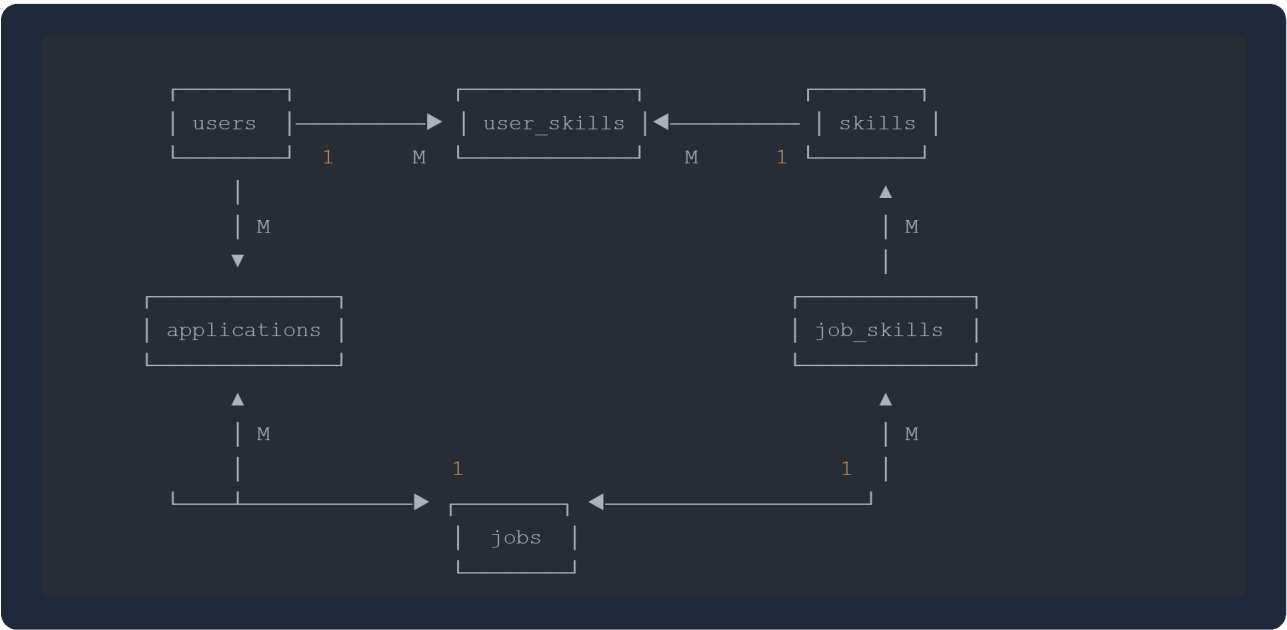
1. Design Philosophy

The system's central design commitment is that **the unit of matching is the user's structured skill profile, not a free-text search query**. Every downstream design choice — the schema's junction tables, the NLP pipeline's gazetteer, the vector-space scoring — exists to serve that one commitment: relevance should be computed, not searched for.

2. Database Design

2.1 Entity-Relationship Overview

Six core entities, two many-to-many relationships mediated by junction tables:



2.2 Table Summary

Table	Role	Key Attributes
users	Registered job seekers	email (unique), password_hash, location fields
skills	Canonical, deduplicated skill taxonomy	skill_name (unique), skill_category
jobs	Canonical, deduplicated postings	source_platform + source_job_id (unique), dedup_hash (indexed), salary range, is_active
user_skills (junction)	A user's declared skill profile	proficiency_level, years_of_experience
job_skills (junction)	NLP-extracted job requirements	relevance_score (0–1), is_required
applications (junction, stateful)	A user's engagement with a posting	application_status (enum), match_score at time of save

2.3 Full PostgreSQL DDL

```
-- =====
-- Skill-Centric Unified Job Search Aggregator System
-- PostgreSQL Data Definition Language (DDL) Script
-- =====

CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ENUM TYPES
CREATE TYPE proficiency_level_enum AS ENUM (
    'BEGINNER', 'INTERMEDIATE', 'ADVANCED', 'EXPERT'
);

CREATE TYPE employment_type_enum AS ENUM (
    'FULL_TIME', 'PART_TIME', 'CONTRACT', 'INTERNSHIP', 'FREELANCE'
);

CREATE TYPE application_status_enum AS ENUM (
    'SAVED', 'APPLIED', 'INTERVIEWING', 'OFFERED', 'REJECTED', 'WITHDRAWN'
);

-- TABLE: users
CREATE TABLE users (
    user_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    full_name        VARCHAR(150) NOT NULL,
    email            VARCHAR(255) NOT NULL UNIQUE,
    password_hash    VARCHAR(255) NOT NULL,
    location_city     VARCHAR(100),
    location_region   VARCHAR(100),
    location_country  VARCHAR(100),
    is_active        BOOLEAN      NOT NULL DEFAULT TRUE,
```

```

        created_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
        updated_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
        CONSTRAINT chk_users_email_format
            CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
    );
CREATE INDEX idx_users_email ON users (email);

-- TABLE: skills
CREATE TABLE skills (
    skill_id            SERIAL PRIMARY KEY,
    skill_name          VARCHAR(120) NOT NULL UNIQUE,
    skill_category      VARCHAR(80),
    created_at          TIMESTAMPTZ    NOT NULL DEFAULT now()
);
CREATE INDEX idx_skills_name ON skills (skill_name);
CREATE INDEX idx_skills_category ON skills (skill_category);

-- TABLE: jobs
CREATE TABLE jobs (
    job_id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_platform     VARCHAR(50)    NOT NULL,
    source_job_id       VARCHAR(255)   NOT NULL,
    job_title           VARCHAR(255)   NOT NULL,
    company_name        VARCHAR(255)   NOT NULL,
    location_city       VARCHAR(100),
    location_region     VARCHAR(100),
    location_country    VARCHAR(100),
    is_remote           BOOLEAN        NOT NULL DEFAULT FALSE,
    employment_type     employment_type_enum,
    job_description     TEXT            NOT NULL,
    job_url             TEXT            NOT NULL,
    salary_min          NUMERIC(12,2),
    salary_max          NUMERIC(12,2),
    salary_currency     CHAR(3),
    posted_date         DATE,
    scraped_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    dedup_hash          CHAR(64)       NOT NULL,
    is_active           BOOLEAN        NOT NULL DEFAULT TRUE,
    created_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    updated_at          TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT uq_jobs_source_platform_id UNIQUE (source_platform, source_job_id),
    CONSTRAINT chk_jobs_salary_range
        CHECK (salary_max IS NULL OR salary_min IS NULL OR salary_max >= salary_min)
);
CREATE INDEX idx_jobs_dedup_hash ON jobs (dedup_hash);
CREATE INDEX idx_jobs_company ON jobs (company_name);
CREATE INDEX idx_jobs_location ON jobs (location_city, location_region, location_country);
CREATE INDEX idx_jobs_posted_date ON jobs (posted_date DESC);
CREATE INDEX idx_jobs_is_active ON jobs (is_active) WHERE is_active = TRUE;

-- TABLE: user_skills (junction)
CREATE TABLE user_skills (
    user_id             UUID NOT NULL,
    skill_id            INTEGER NOT NULL,
    proficiency_level    proficiency_level_enum NOT NULL DEFAULT 'INTERMEDIATE',
    years_of_experience  NUMERIC(4,1) DEFAULT 0,

```

```

        created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
        PRIMARY KEY (user_id, skill_id),
        CONSTRAINT fk_user_skills_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
        CONSTRAINT fk_user_skills_skill FOREIGN KEY (skill_id) REFERENCES skills (skill_id) ON DELETE CASCADE,
        CONSTRAINT chk_user_skills_experience CHECK (years_of_experience >= 0)
    );

CREATE INDEX idx_user_skills_skill_id ON user_skills (skill_id);

-- TABLE: job_skills (junction)
CREATE TABLE job_skills (
    job_id          UUID NOT NULL,
    skill_id         INTEGER NOT NULL,
    relevance_score  NUMERIC(4,3) NOT NULL,
    is_required      BOOLEAN NOT NULL DEFAULT TRUE,
    extracted_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (job_id, skill_id),
    CONSTRAINT fk_job_skills_job FOREIGN KEY (job_id) REFERENCES jobs (job_id) ON DELETE CASCADE,
    CONSTRAINT fk_job_skills_skill FOREIGN KEY (skill_id) REFERENCES skills (skill_id) ON DELETE CASCADE,
    CONSTRAINT chk_job_skills_relevance_range CHECK (relevance_score >= 0 AND relevance_score <= 1)
);

CREATE INDEX idx_job_skills_skill_id ON job_skills (skill_id);
CREATE INDEX idx_job_skills_relevance ON job_skills (relevance_score DESC);

-- TABLE: applications (junction, stateful)
CREATE TABLE applications (
    application_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id            UUID NOT NULL,
    job_id             UUID NOT NULL,
    application_status application_status_enum NOT NULL DEFAULT 'SAVED',
    match_score         NUMERIC(4,3),
    applied_at         TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT fk_applications_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
    CONSTRAINT fk_applications_job FOREIGN KEY (job_id) REFERENCES jobs (job_id) ON DELETE CASCADE,
    CONSTRAINT uq_applications_user_job UNIQUE (user_id, job_id),
    CONSTRAINT chk_applications_match_score_range
        CHECK (match_score IS NULL OR (match_score >= 0 AND match_score <= 1))
);

CREATE INDEX idx_applications_user_id ON applications (user_id);
CREATE INDEX idx_applications_job_id ON applications (job_id);
CREATE INDEX idx_applications_status ON applications (application_status);

```

2.4 Why Junction Tables (not arrays or embedded documents)

Vs. comma-separated/array columns: breaks first normal form, no FK-enforced integrity, no efficient indexed "who knows PostgreSQL" queries.

Vs. document-embedded skills: sacrifices relational joins/aggregates (e.g. "how many postings need this skill") at the DB layer.

Vs. duplicating skill strings per row: guarantees inconsistent casing/spelling across records, defeating exact-match comparison.

Both junction tables are *enriched*, not just structural: `user_skills.proficiency_level / years_of_experience` and `job_skills.relevance_score` are facts about the **relationship**, not about either entity alone — they'd be meaningless attached anywhere else.

3. API Design

3.1 Principles

Versioned REST API (`/api/v1/...`), served by FastAPI, with auto-generated OpenAPI schema.

Every feed item returns *why* it ranked where it did (overlapping/missing skills), never a bare score.

Read-heavy endpoints (`/feed`) are cache-friendly; write endpoints (`/applications`) are synchronous and transactional.

3.2 Endpoint Catalog (v1)

Method	Path	Purpose
POST	<code>/api/v1/auth/register</code>	Create a user account
POST	<code>/api/v1/auth/login</code>	Authenticate, issue session/JWT
GET	<code>/api/v1/users/me</code>	Fetch current user profile
GET	<code>/api/v1/skills</code>	List/search canonical skill taxonomy
POST	<code>/api/v1/users/me/skills</code>	Add/update a skill on the user's profile (proficiency, years)
DELETE	<code>/api/v1/users/me/skills/{skill_id}</code>	Remove a skill from profile
GET	<code>/api/v1/jobs/{job_id}</code>	Fetch a single canonical job posting, incl. extracted skills
GET	<code>/api/v1/feed</code>	Core endpoint — ranked, personalized, deduplicated job feed
POST	<code>/api/v1/applications</code>	Save/track a job (status defaults to <code>SAVED</code>)
PATCH	<code>/api/v1/applications/{application_id}</code>	Update application status (<code>APPLIED</code> , <code>INTERVIEWING</code> , etc.)
GET	<code>/api/v1/applications</code>	List the user's tracked applications

GET /feed query parameters: limit, offset, location, employment_type, is_remote, min_match_score — all filters compose with (not replace) the semantic ranking.

4. Core Algorithm Design

4.1 Deduplication Heuristic (Standardization Stage)

A candidate pair is flagged a duplicate **only if all three** conditions hold simultaneously:

Field	Method	Threshold
Company name	Normalized exact match + Levenshtein ratio	≥ 0.90
Job title	Token-set fuzzy similarity (Levenshtein/Jaccard)	≥ 0.85
Location	Normalized exact match on city + region	exact

Fast path: `dedup_hash = SHA256(normalized_company + normalized_title + normalized_location)`, indexed on `jobs`, catches exact duplicates in $O(1)$ before the more expensive fuzzy comparison runs. The record with the most complete metadata (longest description, most recent `posted_date`) is retained as canonical; the other's platform/URL is preserved as an alternate listing reference so the user can still apply through their preferred platform.

4.2 Skill Extraction — Phase A

Tokenize + POS-tag the job description; isolate noun phrases.

Cross-reference phrases against a curated skill-taxonomy gazetteer using exact-match and fuzzy-match resolution (spaCy custom NER component).

Compute `relevance_score` as a weighted combination of:

- Frequency of the skill within the description.

- Proximity to requirement-signaling language ("required", "must have", "must possess").

- Section position (skills in a "Requirements" heading are weighted above passing mentions).

Persist to `job_skills (job_id, skill_id, relevance_score, is_required)`.

4.3 Semantic Matching — Phase B

Represent both a user's `user_skills` set and a job's `job_skills` set as TF-IDF vectors over the skill taxonomy.

Compute `match_score = cosine_similarity(user_vector, job_vector)` .

This is deliberately **not** simple set intersection: a user skilled in React and Vue.js should register partial (not zero) similarity against a posting requiring Angular, because the vector space captures skill co-occurrence patterns, not just exact overlap.

v1 upgrade path: replace TF-IDF vectors with dense transformer sentence embeddings without changing the storage schema or API contract — only the vectorization step changes.

5. Frontend / UX Design

5.1 Key Screens

Screen	Purpose
Register / Login	Account creation and auth
Skill Profile	Add/edit skills, proficiency, years of experience
Personalized Feed	Ranked job cards, filterable, each showing match % + overlapping/missing skills
Job Detail	Full description, source platform link(s), save/apply-tracking controls
Application Tracker	SAVED → APPLIED → INTERVIEWING → OFFERED/REJECTED/WITHDRAWN

5.2 Component Notes

Feed cards are the highest-traffic component — should be virtualized/paginated given a growing job corpus.

Match explanation (overlapping vs. missing skills) is a first-class UI element, not a tooltip afterthought — it's core to the product's differentiation from keyword search.

Next.js SSR is used specifically for the feed's initial paint, given it's the primary landing surface.

6. Key Design Trade-offs

Decision	Alternative Considered	Why Rejected (for v1)
TF-IDF cosine similarity	Transformer embeddings from day one	Heavier compute/infra for a PoC; TF-IDF is a clean, swappable v1 with a defined upgrade path
Composite fuzzy dedup heuristic	ML-based duplicate classifier	Higher implementation cost for marginal PoC-stage gain; heuristic is explainable and tunable
Staging table before canonical write	Direct upstream-to-canonical write	Removes blast radius of upstream format changes; enables reprocessing without data loss
Async Phase A extraction	Synchronous extraction on ingestion	Keeps ingestion throughput decoupled from CPU-bound NLP cost

Companion documents: `Architecture.md` (system structure), `phase.md` (delivery plan), `rules.md` (engineering rules), `memory.md` (decision log).